GovGraph

Client Handover Document

COMP3900 Capstone Project, 2026 T2 — Team T18B-BANANA
University of New South Wales (UNSW)

A note to Ursula and the UNSW team

Dear Ursula, and everyone at UNSW involved in this project,

Thank you for the opportunity to work on GovGraph this term. It has been a rewarding project for our team, and we're proud of what we've built together. This document is written for whoever takes GovGraph forward from here — it explains what the system does, how it's put together, and what to know before making changes.

We've tried to keep this handover in plain language rather than heavy technical diagrams, so that anyone on the client's team can read it, not just developers. Where we'd normally draw a diagram, we've described it in words instead.

If anything here is unclear, or you'd like a walkthrough of any part of the system, please don't hesitate to reach out to the team (contact details in the final section of this document).

Thank you again for your guidance and support throughout the project.

Kind regards,

The T18B-BANANA Team

1. What this document is for

This is the handover reference for the team taking over GovGraph after the capstone project ends. It covers:

- How the system is built and why we made the choices we did.
- What each feature does, and how finished it is.
- What to know about the AI duplicate-detection feature, including a few things worth improving.
- How to run the system on a computer.

This document is meant to be read alongside the technical README files in the project (the main README, and one each for the backend and frontend code), which cover the exact commands to run the system day to day.

2. What GovGraph is

GovGraph is a website that acts as a single source of truth for a university's governance structures — that is, its committees, boards, working groups, and advisory panels. It lets staff:

- Register a new governance group by filling out a guided Terms-of-Reference (ToR) form, which then goes to an administrator for review and approval.
- Keep a registry of all groups, who belongs to them, and what role each person holds (Chair, Secretariat, Member, and so on).
- See how groups relate to one another — for example, which groups report to which other groups — as an interactive diagram they can pan, zoom, and rearrange.
- Manage the underlying reference lists (types of groups, types of roles, types of relationships) themselves, without needing a developer to change the code.

3. How the system is built

GovGraph is made up of three main parts that work together:

- A website (the “frontend”) that runs in the user's browser and is what staff actually see and click on. It's built with React, a popular tool for building interactive websites.
- A server (the “backend”) that the website talks to behind the scenes. It handles logic like “is this user allowed to approve this group?” and “what groups exist right now?” It's built in Python using a framework called Flask.
- A database, which is where all the actual data lives permanently — every group, person, membership, and relationship. We use PostgreSQL, a well-established, free database system.

In plain terms: when a staff member clicks something on the website, the website sends a request to the server, the server checks the rules and talks to the database if it needs to read or

save something, and then sends a response back to the website to display. All of this happens over a secure web connection.

Why we chose these tools

Part	What we used	Why
Frontend (website)	React with TypeScript, plus a library called React Flow for the group diagram	React Flow gives us interactive dragging, zooming, and panning for the governance diagram 'for free', which would otherwise have taken a lot of custom work to build from scratch.
Backend (server)	Python with Flask	Flask is lightweight and easy for a new developer to understand quickly, which matters for a handover like this one.
Database	PostgreSQL for real use; a lightweight in-memory database for automated tests	PostgreSQL is reliable and free to run. The lightweight version used only during testing makes the automated tests run fast, without needing a full database set up just to test the code.
Logins and permissions	A simple two-level system: 'Staff' and 'Super User'	This matches how the client actually works: general staff submit and view information, while a smaller group of super users review and approve new governance groups.
Automated checks	GitHub Actions, which automatically checks the code every time someone proposes a change	This catches mistakes before they're merged into the main codebase, without needing a separate test server running all the time.

Where things live in the code (for developers)

For whoever picks up the codebase, at a high level:

- The backend code lives in a folder called backend/app. Inside it, db_schema holds the definitions of every database table, routes holds the web addresses (endpoints) the website can call, and services holds the actual business logic for each area — authentication, registration, groups, people, Terms of Reference, the diagram, and the AI similarity-search feature.
- The frontend code lives in frontend/src. It has a folder for API calls to the backend, a folder for login/authentication handling, a folder for the group diagram component, a folder for the multi-step Terms-of-Reference form, and a folder for each full page of the website (login, register, admin registry, new group, registry, and the diagram view).

4. How the data is organised, and why

Full technical detail on every database table is in backend/app/db_schema/TABLES.md. The choices worth explaining to the client are:

- Group types, role types, and relationship types are stored as editable lists in the database, not hard-coded into the program. This means the client can add a new committee type, or a new role such as 'Treasurer', themselves — without needing a developer to change and redeploy the code.
- Whether a relationship has a direction (for example, 'Reports to' has a direction, but 'Works with' doesn't) is stored on the relationship type itself, not on each individual relationship. This keeps that behaviour consistent everywhere in the system.
- Every person and every group has a public reference code (a UUID) separate from its internal database ID. This matters for the approval process: when a submitted form is approved and becomes a live group, it keeps the same public reference code it had as a submission, so the two are always traceable to each other.
- A submitted Terms-of-Reference form is stored separately from the live governance group it eventually creates. The submission record keeps the raw form data and its review status (draft, pending review, approved); the live group record is only created once a submission is approved. This keeps the review process, which changes often, separate from the core registry data that other features rely on staying stable.
- A person's membership in a group is stored separately from the specific roles they hold in it, because one person can hold more than one role in the same group at the same time — for example, being both Chair and Treasurer.
- External reference links (such as a link to the ToR document, a records-management entry, or meeting minutes) are stored as a list attached to each group, rather than as fixed fields, because a group can have any number of these links.

5. What's finished, feature by feature

Feature	Status	Notes
Logging in, roles, and registration	Done	Staff can register and log in immediately. Super User registration needs an administrator's approval first.
Admin review of new user registrations	Done	Handled on the Admin Registry page in the website.
Terms-of-Reference submission form	Done	A guided, multi-step form that walks staff through registering a new group.
Terms-of-Reference review and approval	Done	Submissions move from draft, to pending review, to approved, with admin comments along the way.
Governance registry (groups, people, memberships, roles)	Done	Full create/view/update/delete support.
Governance relationship	Done	Interactive, drag-and-zoom diagram of how

diagram		groups relate to each other.
Editable reference lists (group types, roles, relationships)	Done	The client can change these themselves without a code change.
API documentation	Done	A browsable reference of every backend web address is available at /api/docs once the system is running.
Automated checks on every code change	Done	Runs automatically via GitHub Actions.
AI-based duplicate group detection	Done	Explained in full in Section 6, including a few things worth improving.

One small note: the backend's own README file has a table describing some web addresses as unfinished 'skeletons' from early in the project. That table is now out of date — those features are finished. We'd recommend the client's team refresh that table so it isn't misleading to a future developer.

6. The AI duplicate-group detection feature

One of the more interesting pieces of GovGraph is a feature that automatically warns staff when a newly submitted governance group looks like it might already exist under a different name. This started as a small side experiment to see if the idea would even work, and has since been built properly into the real system.

6.1 How it works, described in words

Rather than drawing this as a flowchart, here is the process in plain steps:

1. When a Terms-of-Reference form is submitted, the system pulls out just the parts that actually describe what makes the group distinct: its type, its purpose, its responsibilities, and which group it reports to. Everything else in the form (dates, member names, standard boilerplate wording) is left out at this stage.
2. That short, distilled description is turned into a set of numbers called an 'embedding', using a local AI model. This is a standard technique for comparing how similar two pieces of text are in meaning, not just in wording. This step runs on our own server, so it has no ongoing cost per use.
3. The system compares this new embedding against the embeddings already stored for every existing group, and works out a similarity score for each. Anything scoring above a threshold we calibrated (0.55 out of 1.0) is treated as a possible match.
4. Because that first pass can occasionally flag two groups as similar just because they use similar generic committee language, every possible match is then double-checked by sending both groups' descriptions to Google's Gemini AI model, and asking it to judge — based on what the groups actually do, not just how they're worded — whether they are genuinely the same or overlapping group.

5. Only the matches Gemini agrees are genuine are shown to staff, as a warning that lists the name(s) of the group(s) it resembles.

6.2 The one important lesson learned

Every Terms-of-Reference document in the client's dataset is generated from the same template, so a large majority of the wording (the standard sections on purpose, membership, quorum, and so on) is nearly identical across every single group. Early on, we tried comparing the full raw documents to each other directly, and found that this made almost every pair of groups look like a duplicate, because the AI model was picking up on the shared template wording rather than what actually made each group different.

The fix was to only compare the distilled, distinguishing information described above — never the raw document. This is an important detail for whoever maintains this feature in future: if someone changes the code to compare full documents again, duplicate detection will silently stop being useful, because everything will look like a duplicate of everything else.

We also deliberately chose a two-step approach (a quick automatic first pass, followed by a more careful AI check) rather than relying on either step alone. Missing a real duplicate is a bigger problem than flagging something that turns out not to be one, since a duplicate group that slips through and goes live is hard to undo later, while a false alarm just costs one extra, inexpensive AI check.

6.3 What's built, and where to find it

This feature is fully working in the live system, not just a prototype:

- Every group's distinguishing description and its embedding are calculated once, when the group is submitted, and stored permanently alongside the group's other data. They are not recalculated every time — the comparison only needs to run once per new submission.
- There is a dedicated web address the system can be asked to check for duplicates on demand, separate from actually submitting a group. Submitting a group also runs this same check automatically and includes the result in its response.
- The website shows a small amber warning badge in the group registry when a group looks similar to another one, listing which group(s) it resembles.
- This AI similarity check is separate from a simpler check the system already had, which simply blocks two groups from having the exact same name. Both checks run; they catch different kinds of problems.
- There is automated test coverage for both the similarity-scoring step and the AI verification step.

6.4 A few things worth improving

The feature works and is genuinely useful today, but there are three things the client's team may want to look at next:

- Right now, the AI double-check happens while the user is waiting for the page to respond, which adds a small delay (and a small cost) to every submission. This could be moved to run in the background instead, so the response is instant and the double-check result appears a moment later.
- The warning currently only appears afterwards, on the registry page where an administrator reviews submissions — not to the staff member while they are still filling out the form. Showing the same warning earlier, during the submission form itself, would let people catch the problem before they even submit.
- This feature needs an active Google Gemini account key to do the careful AI double-check. If that key isn't set up, the system doesn't break — it simply skips the double-check and shows the first-pass matches without that extra layer of confidence.

6.5 The original experiment

The very first version of this idea still exists in the project as a separate, standalone set of scripts (in a folder called similarity_search), kept as-is. It's no longer used by the live website, but it's still useful on its own as an offline tool — for example, to re-check the calibration if the client's dataset changes significantly, or to pull the same distilled information out of a batch of Word documents outside of the main system. One thing to flag: that tool reads specific, numbered section headings (Purpose, Reporting, Scope) from the current Terms-of-Reference template. If the client ever changes that template's structure, both this standalone tool and the equivalent logic in the live system will need to be updated to match, or the automatic extraction will quietly misread the document.

7. How to run the system

Full, exact instructions are in the main README file at the root of the project. In short, running GovGraph on a computer involves three steps: start the database, start the backend server, and start the frontend website. Once all three are running, the website is available in a browser, and it talks to the backend automatically.

The project also has a built-in way to double-check that the code is healthy: running the backend's automated tests and code-style checks, and running the frontend's equivalent checks and build process. These are the same checks that run automatically on every proposed code change.

8. Team and contacts at handover

Name	Role
Jinhao Li	Product Owner + Backend
ZhengYu Chen	Backend + Database
Riyaz Mohamed Rabeek	Scrum Master + Backend
Raymond Zhuang	Frontend

Tim Faithfull	Backend
Johny Ha	Frontend